

High-Performance Data Analysis

(Análise de Dados de Elevado Desempenho)

Introduction to Distributed Machine Learning

University of Minho

2025-2026



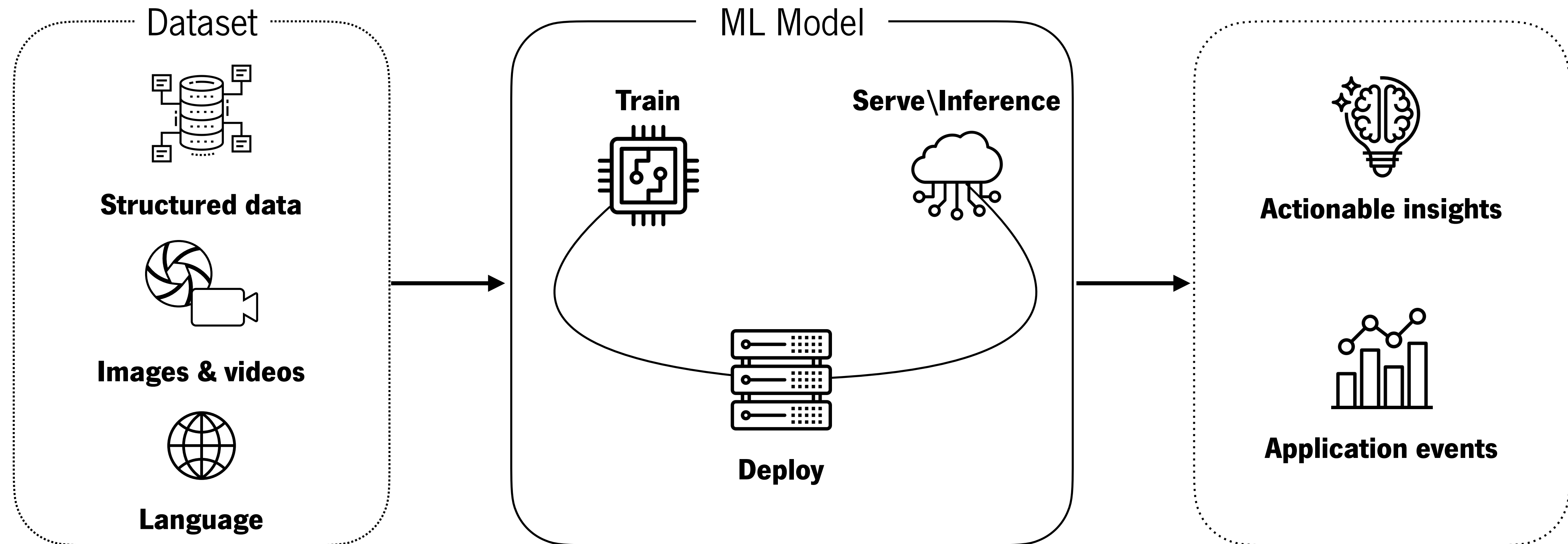
Machine Learning

“A computer program is said to learn from experience E with respect to some task T and some performance measure P , if its performance on T , as measured by P , improves with experience E .”

● E.g.,

- ❖ Experience - the user is classifying its own emails as spam or not spam
- ❖ Task - classification of emails
- ❖ Performance - Correct or incorrect classification of the emails

Machine Learning Life Cycle



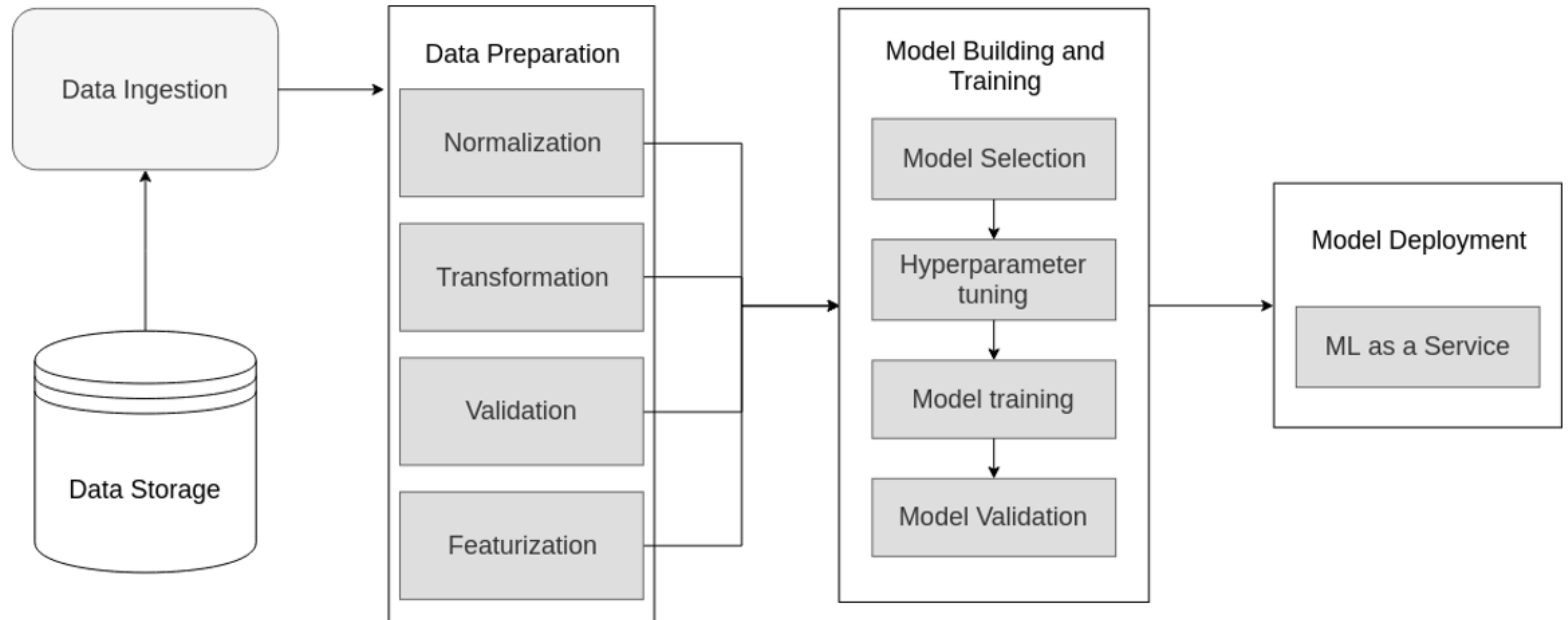
<https://www.qlik.com/us/augmented-analytics/machine-learning-vs-ai>

Machine Learning Pipeline

(Distributed) Data Processing (Apache Spark)

(Distributed) Training

(Distributed) Serving/
Inference



Adapted from Microsoft Azure: <https://docs.microsoft.com/en-us/azure/machine-learning/service/concept-ml-pipelines>

Learning Methods

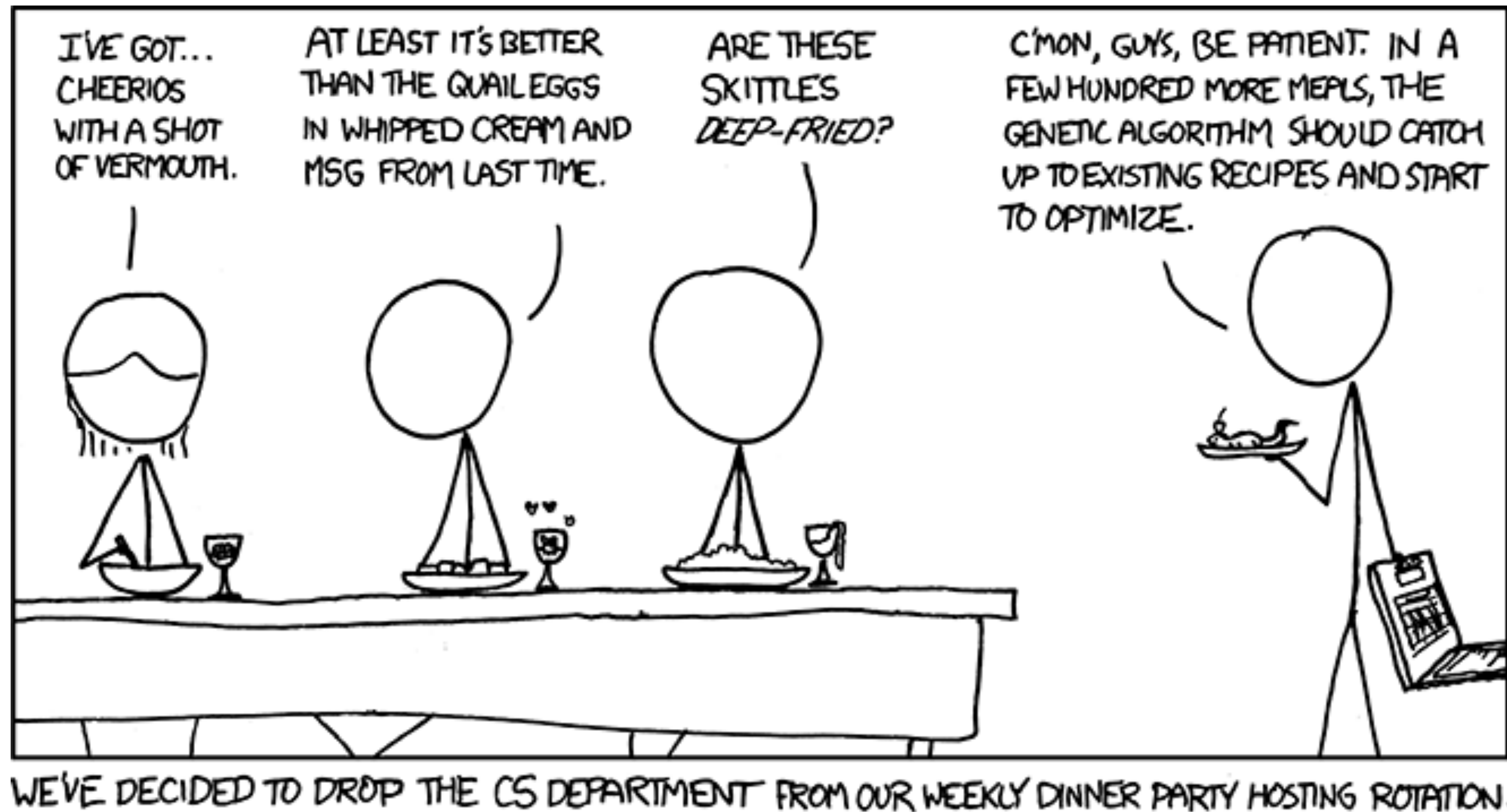
- **Supervised Learning** - data is labeled
- **Unsupervised Learning** - data is structured
- **Semi-supervised Learning** - data is partially labeled and has structure
- **Reinforcement Learning** - based on rewards, good actions offer better rewards, and bad actions give negative rewards

Concepts

Dataset	Function
TRAINING	MODEL IS TRAINED TO FIT THIS
VALIDATION	MODEL IS REPEATEDLY TESTED ON THIS DATA DURING EVALUATION
TESTING	MEASURE PERFORMANCE OF A TRAINED MODEL

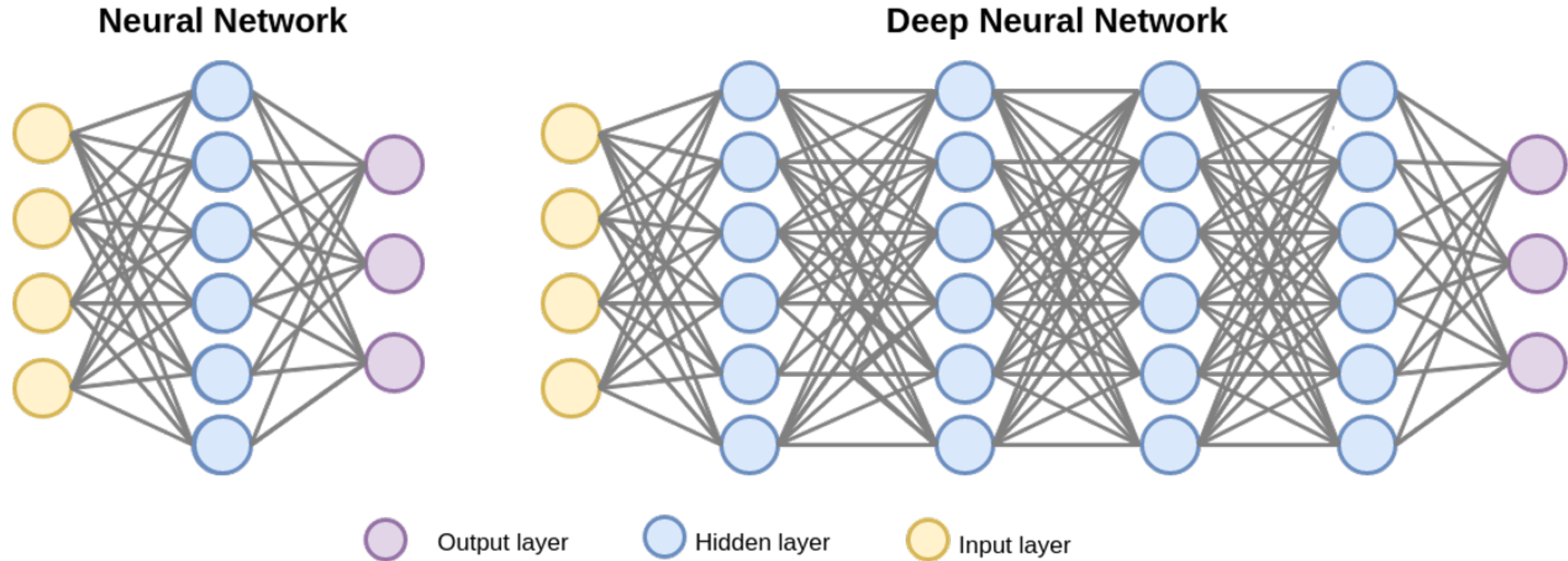
- **Batch size:** size of a subset of a dataset (how the dataset is split into chunks)
- **Step/Iteration:** one forward+backward+update using one batch
- **Epoch:** one pass through the full training dataset
- **Steps per epoch:** $N_{\text{train}} / \text{batch_size}$

Deep Learning



<https://xkcd.com/720/>

Neural Networks

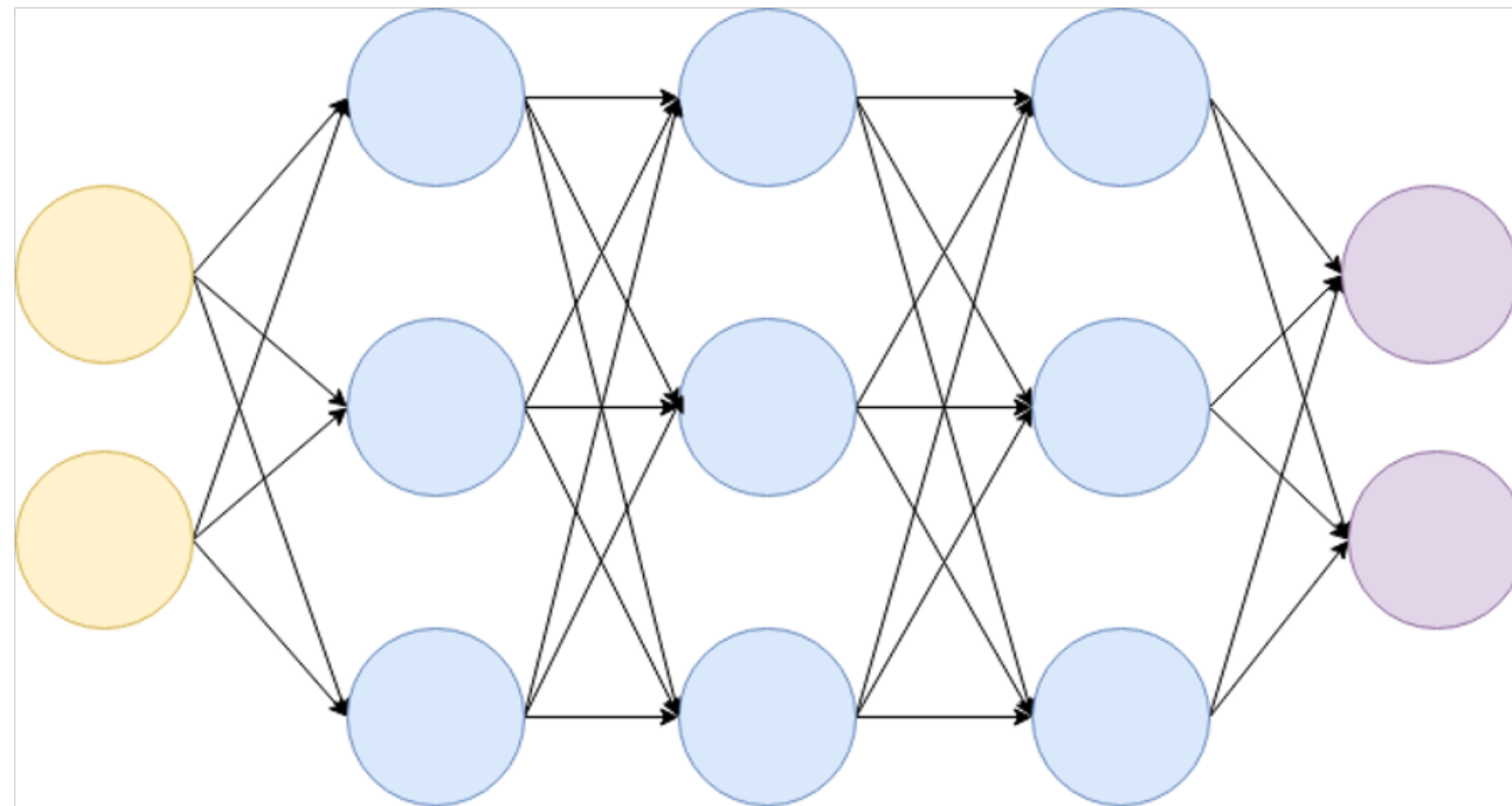


Training = repeat forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ update (SGD)

Deep Neural Network (DNN)

- ◎ **Parameters** → weights (+ bias) that SGD updates.
- ◎ **Layers** → Different layer types exist... but DML cares about compute + memory + communication.
- ◎ **Activation Functions** → Activation functions add non-linearity (ReLU most common).
- ◎ **Loss Functions** → Regression: MSE (L2); Classification: Cross-entropy.
- ◎ **Optimizers** → SGD (baseline), Adam (common default)
 - ▶ In DML, we typically address SGD-like updates (Distributed SGD).
- ◎ **Hyperparameters**
 - ▶ The learning rate often changes with the global batch size.
 - ▶ Batch size is limited by memory; **distribution changes this.**

Feed-Forward Propagation



$$y_i = \sigma\left(W_{o,i} + \sum_{j=1}^d Z_j W_{i,j}\right)$$
$$J(W) = \frac{1}{n} \sum_{i=1}^n L(f(x^{(i)}; W), y^{(i)})$$

A curved arrow points from the y_i term in the first equation to the $f(x^{(i)}; W)$ term in the second equation.

Compute-heavy step

$n \rightarrow$ number of samples in the batch/dataset

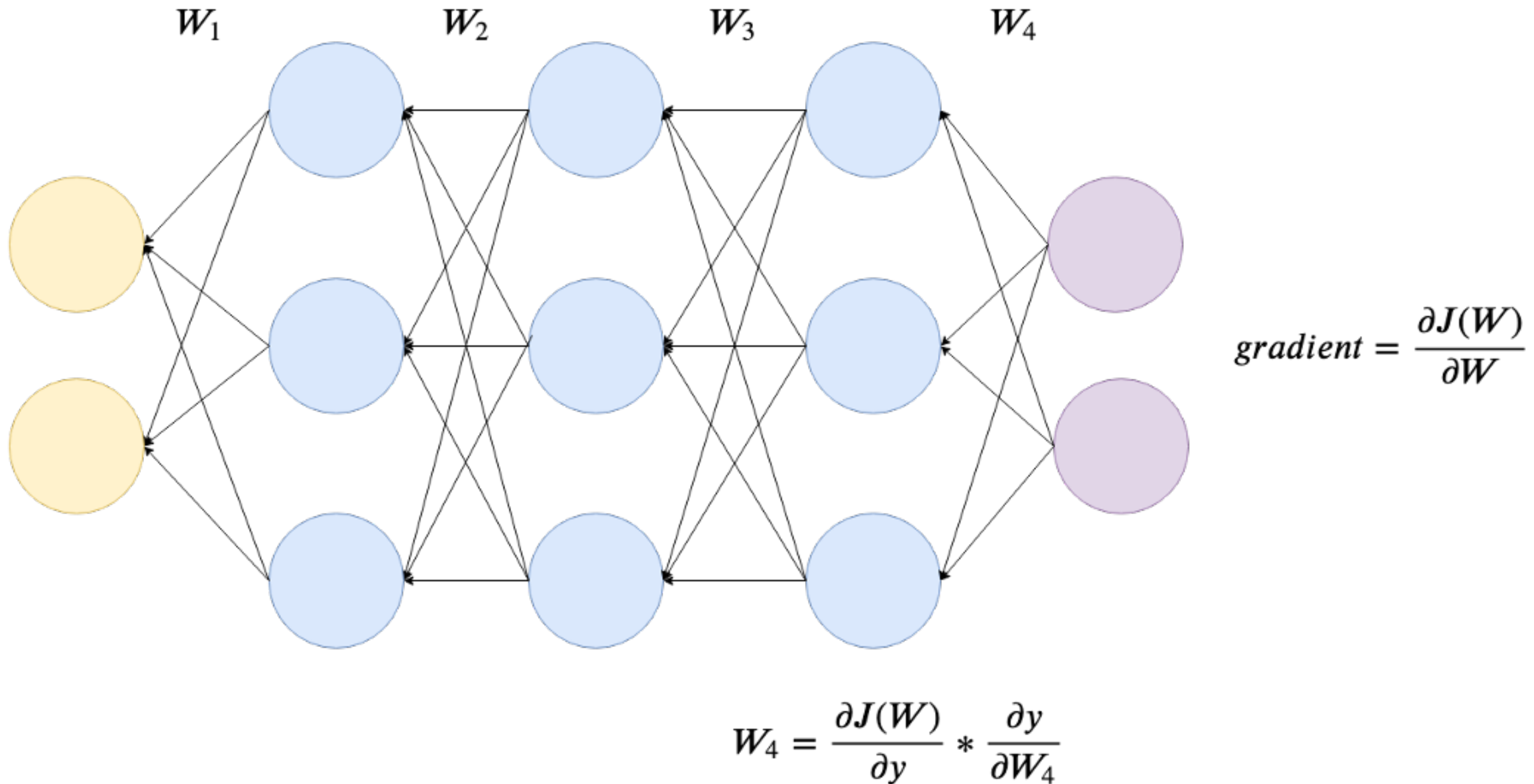
Feed-Forward Propagation

$$W^* = \operatorname{argmin}(\frac{1}{n} \sum_{i=1}^n L(f(x^{(i)}; W), y^{(i)}))$$

$$W^* = \operatorname{argmin}(J(W))$$

Distributed training = compute parts of this sum in parallel.

Backward Propagation



Backward Propagation

Algorithm 1 Gradient Descent

```
1: Initialise weights randomly
2: while not convergence do
3:   Compute gradient,  $\frac{\partial J(W)}{\partial W}$ 
4:   Update weights,  $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$ 
5: return Weights
```

Algorithm 2 Stochastic Gradient Descent

```
1: Initialise weights randomly
2: while not convergence do
3:   Pick batch of B data points
4:   Compute gradient,  $\frac{\partial J(W)}{\partial W} = \frac{1}{B} \sum_{k=1}^B \frac{\partial J_k(W)}{\partial W}$ 
5:   Update weights,  $W \leftarrow W - \eta \frac{\partial J(W)}{\partial W}$ 
6: return Weights
```

➡ **Gradients usually reduce loss,**
but deep nets are non-convex → local
minima / saddle points.

➡ **Distributed SGD** (next)
**aggregates gradients across
workers.**

Distributed Machine Learning (DML)

◎ What is distributed ML?

- Training (or serving) a model using multiple devices/nodes where computation and/or state is split.

◎ What's being split?

- Compute, data, model state (parameters/optimizer), or inference requests.

◎ What really changes?

- Distributed training mostly changes how gradients are aggregated and parameters updated.
 - As such, in distributed training, gradients are the **payload**; the hard part is communicating/aggregating them efficiently.

Distributed Machine Learning (DML)

- ◎ In the end, why choose/use DML?
 - ▶ **Speed:** reduce time-to-train by using many devices in parallel
 - ▶ **Scale:** models/data don't fit on one device (memory + compute limits)
 - ▶ **Data locality & privacy:** data can't move (edge / federated settings)
 - ▶ **Serving at scale:** throughput + latency constraints (distributed inference)

Distributed Machine Learning Topologies

Based on “Where is data?”, “Who coordinates?”

- ◎ Data Parallelism
- ◎ Model Parallelism
- ◎ Gossip Learning
- ◎ Federated Computation

Distributed Machine Learning Topologies

- ◎ **Data Parallelism** - when data does not fit in a single device, and we want to accelerate the training time. Model is replicated and we split batches.
- ◎ Model Parallelism
- ◎ Gossip Learning
- ◎ Federated Computation

Distributed Machine Learning Topologies

- ◎ Data Parallelism
- ◎ **Model Parallelism** - when the model does not fit on one device, we split the model state.
- ◎ Gossip Learning
- ◎ Federated Computation

Distributed Machine Learning Topologies

- Data Parallelism
- Model Parallelism
- **Gossip Learning** - peer-to-peer training, global communication
- Federated Computation

Distributed Machine Learning Topologies

- Data Parallelism
- Model Parallelism
- Gossip Learning
- **Federated Computation** - a group of entities that compute over the same model, reaching a final consensus on a global model while preserving their local model.

Vocabulary you must not mix up

- **Device:** GPU/TPU (where tensors live and computation happens)
- **Node:** one machine (often multiple devices)
- **Worker/process:** training program instance (one per device or per node)
- **Rank:** worker ID within a distributed job
- **Local batch size (B):** batch per worker
- **Global batch size (B^g):** $B \times (\text{\#workers})$ (or $\times$ accumulation steps)
- **Gradient accumulation:** simulate a larger global batch without more workers

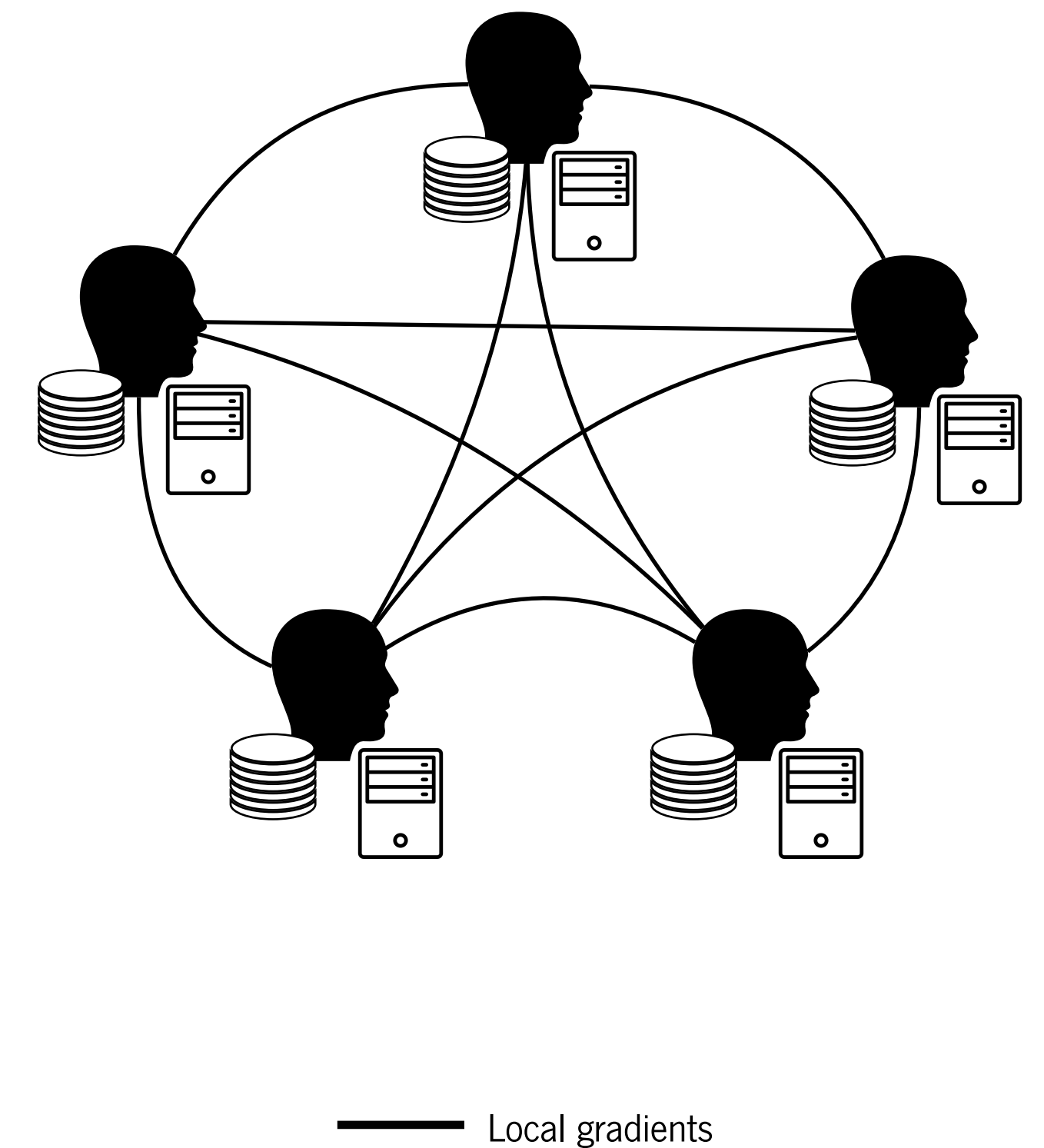
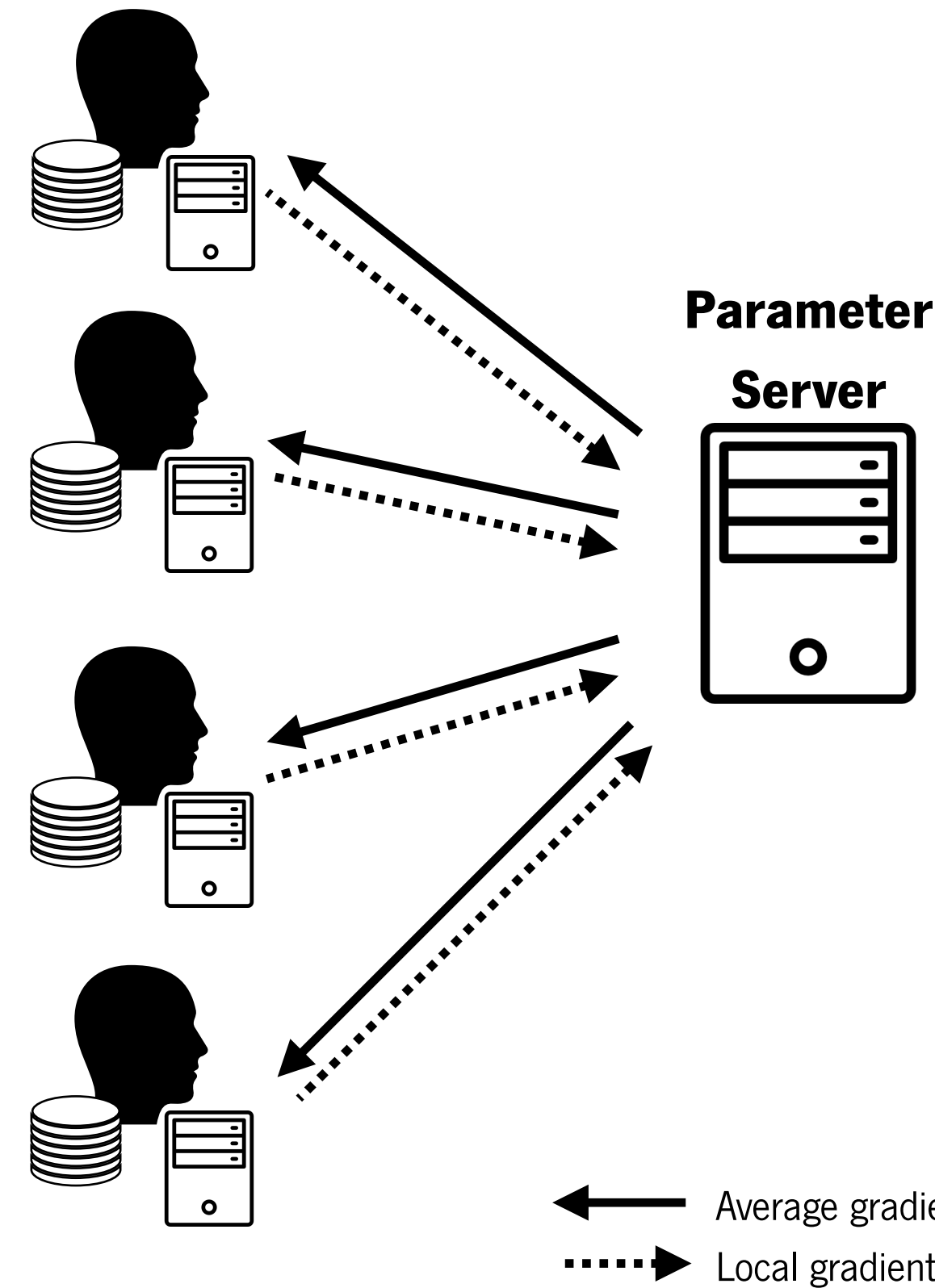
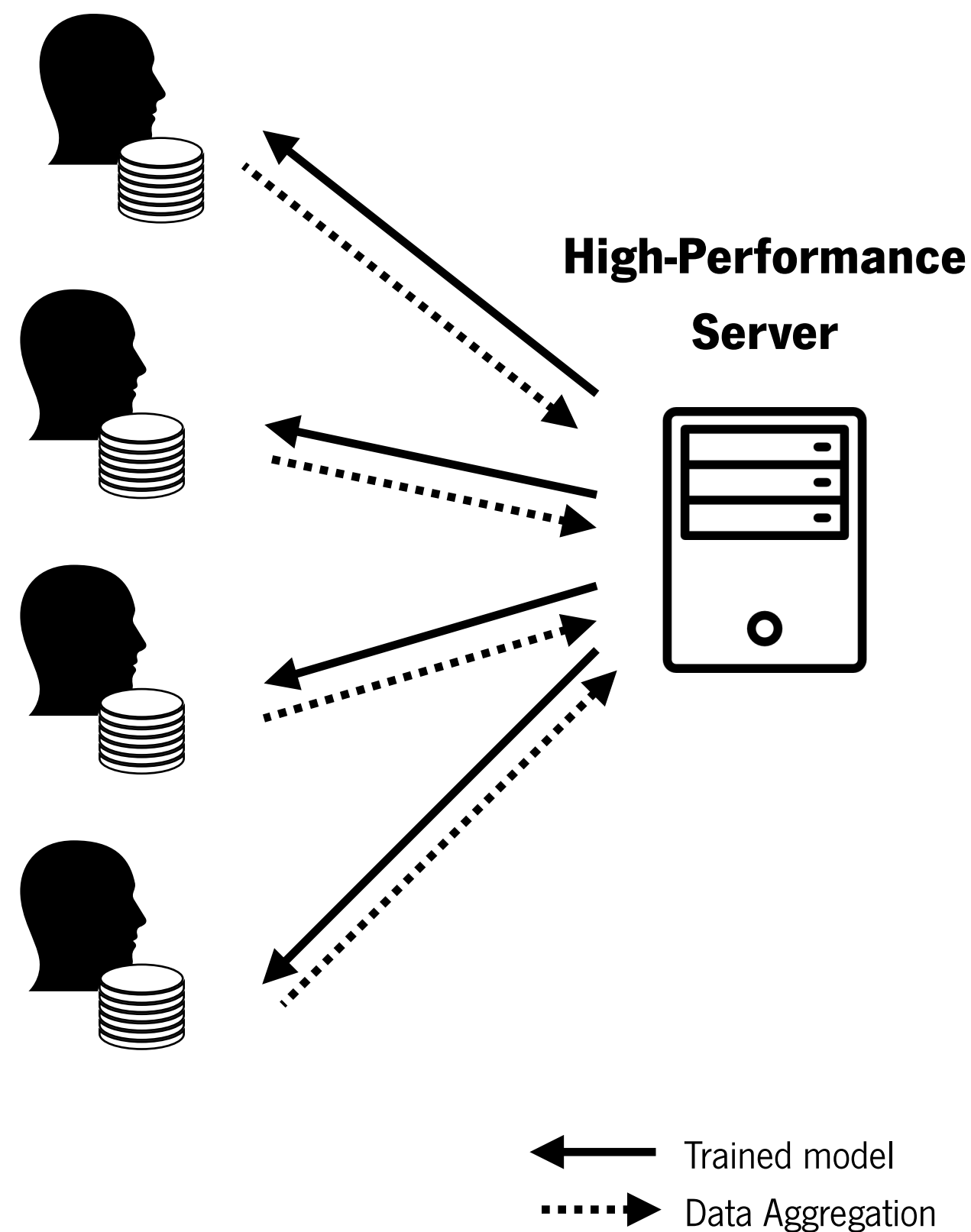
How workers communicate

Centralised training

Parameter Server (centralised coordination)

Decentralized/Gossip training

Clients

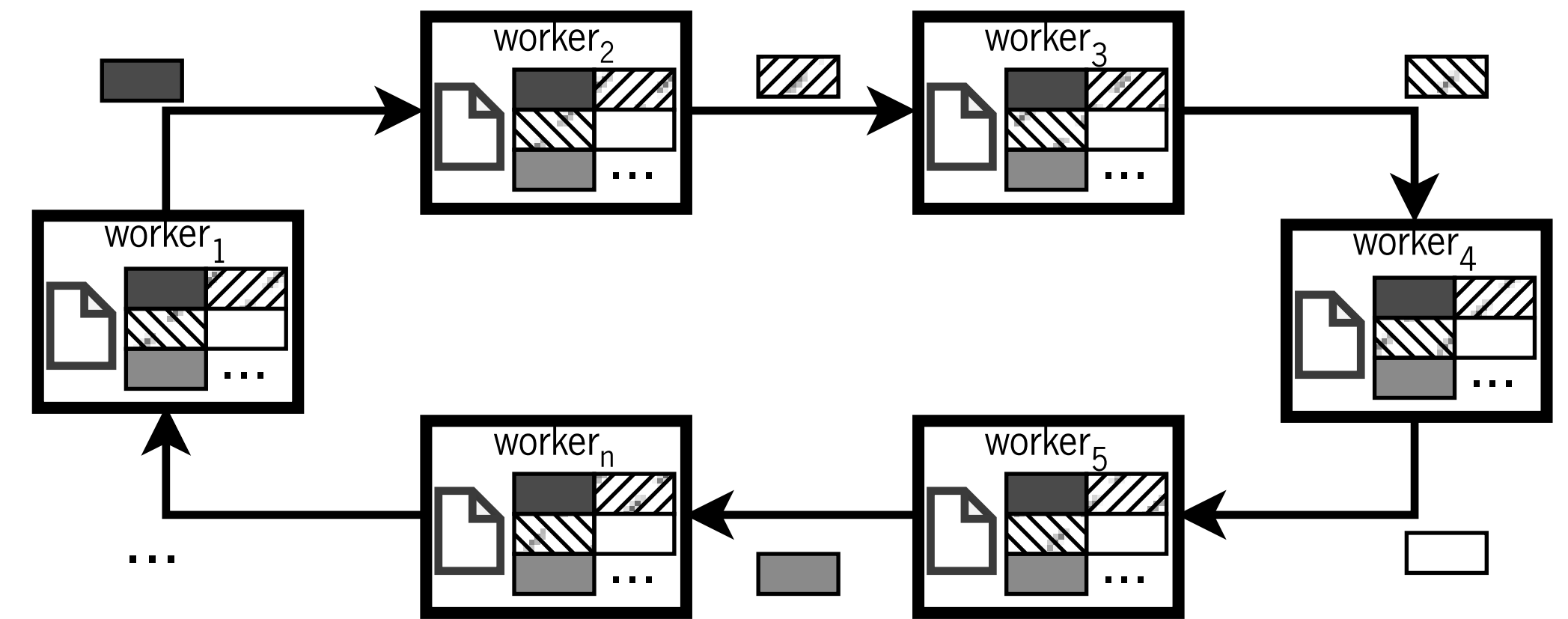


PS vs Gossip vs AllReduce

- ◉ **Parameter Server** - Server that aggregates the gradients from all the workers connected to it. Depending on the approach, it may provide the average of gradients or the sum of all the gradients.
 - **Pros:** Simple coordination, easy elasticity
 - **Cons:** Server bottleneck / hotspot, failure sensitivity
- ◉ **Gossip Learning** - Peer-to-peer mixing of updates, each worker chooses which workers they want to update their gradients with.
 - **Pros:** Robust with no centralised entity
 - **Cons:** Slower to converge and harder to tune
- ◉ **All Reduce:** workers compute gradients → collective average → each updates locally
 - **Pros:** High bandwidth efficiency on fast networks, no single bottleneck
 - **Cons:** Synchronisation overhead, sensitive to stragglers

Distributed SGD (AllReduce) in one picture

- AllReduce offers a way to run SGD in a distributed fashion.
 - The approach involves parallelizing mini-batches across several machines, with weight updates performed afterward.



AllReduce $\approx$ everyone computes gradients $\rightarrow$ collective average $\rightarrow$ everyone applies the same update.

Synchronous vs Asynchronous

- **Synchronous:** All workers compute gradients for step $t \rightarrow$ aggregate $\rightarrow$ update together.
 - Deterministic-ish; easier debugging; common default.
 - Straggler effect (slowest worker sets pace).
- **Asynchronous:** Workers push updates without waiting for others.
 - Better utilization under heterogeneity.
 - Stale gradients; harder to tune / reason about convergence.
- **Tradeoff:** throughput vs stability

Why scaling is not free

When we are distributing computation, the step time changes to:

$$\text{Step time} = \text{compute} + \mathbf{\text{communication}} + \text{I/O}$$

As you add devices:

- compute/worker ↓, but communication + overhead ↑

Possible bottlenecks:

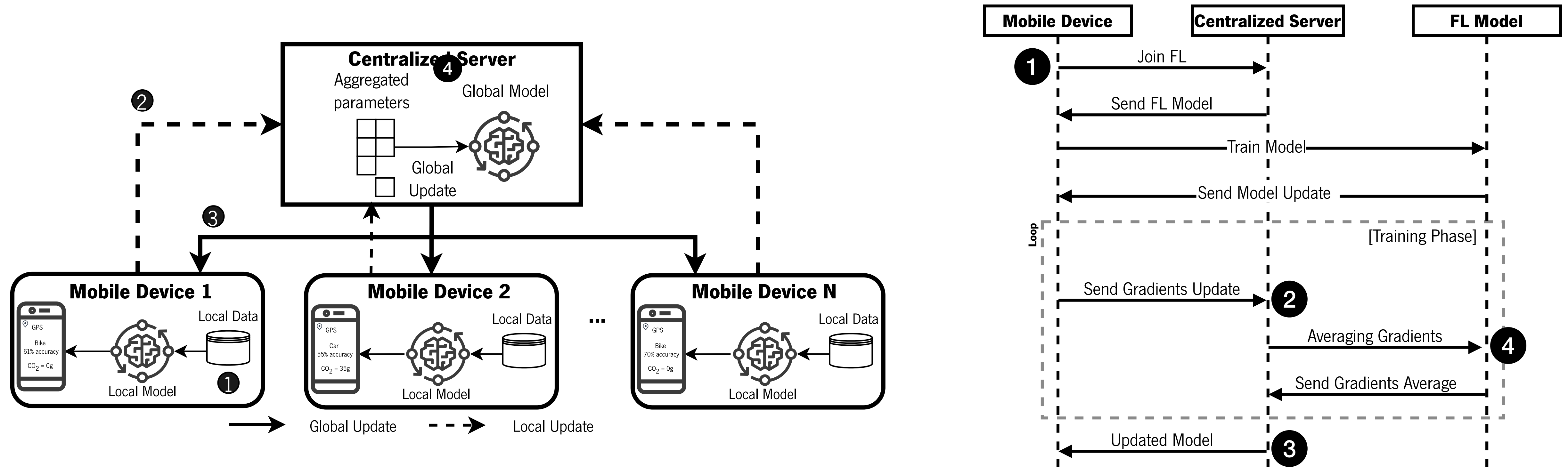
- ⊙ Bandwidth/latency, Stragglers, Memory, Checkpointing

Useful Metrics:

- ⊙ $\text{Speedup} = T_1 / T_p$

- ⊙ $\text{Efficiency} = \text{Speedup} / p$

Federated Learning



- 1) Data remains on client devices, and training happens where data lives
- 2) Server orchestrates rounds: select clients and aggregate updates

Federated Learning vs DML

	Distributed Training	Federated Learning
Aims	Scale model training speed/size.	Processing distributed data on heterogenous data sources.
Datasets	IID (identically and independently distributed) and have roughly the same size.	Heterogenous data sets; their sizes may span several orders of magnitude.
Nodes	Data centers with powerful computing resources connected by high-performance networks.	Low-power devices , such as smartphones and IoT devices, with limited computing resources connected by less reliable networks , which results in their failures or dropping out of computation.
Data Privacy*	Data is usually stored in a shared file system (parallel file system).	Data does not leave the users premises.
Network	Reliable high bandwidth.	Variable low bandwidth

Takeaways

- **Distributed SGD** equals looking at parallel gradients with coordinated updates.
- **Communication patterns matter** (PS / AllReduce / Gossip), whether synchronous or asynchronous.
- Federated learning **greatly differs** from data-center distributed training.

System scaling vs Statistical scaling

- **System scaling:** more devices → lower step time (compute/comm overlap).
- **Statistical scaling:** larger global batch changes the optimization regime.
- **Measure the right thing:** optimize time-to-accuracy, not only time/step.
- **Large batch often needs tuning:** LR schedule / warmup / regularization.
- **Diminishing returns:** communication + imbalance + overhead dominate.

Why LLMs force distributed training

- ◎ LLMs have **billions of parameters** → single device memory is insufficient (params + activations + optimizer state).
- ◎ **Massive data/token counts** → time-to-train
- ◎ Real systems combine **multiple parallelism types** (next class)
- ◎ **Inference also stresses memory/throughput:**
 - weights + KV cache (KV grows with context length),

Questions?